CSC 212: Data Structures and Abstractions

09: Linked Lists

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



From last lecture …

- The circular array made a queue and a deque cheap at **both ends**
  - move the index, not the data: every end operation is $\Theta(1)$ amortized
- Two costs remain
  - insert or erase in the middle: $\Theta(n)$, the data must shift
  - growing: allocate a bigger block, copy everything, $\Theta(n)$ in the worst case
- Today
  - each element lives anywhere in memory and points to the next one: the **linked list**

The snake game

- The classic game: a snake moves on a grid, one cell per tick
  - **move:** the head enters a new cell, the tail leaves its cell
  - **eat:** the head enters a new cell, the tail stays, the snake grows by one
- The body is a sequence of cells, head first
  - number each cell `row * width + col`

```
time t                          time t+1, move right

. . . . . . . .                 . . . . . . . .
. o o o @ . . .                 . o o o o @ . .
. o . . . . . .                 . . . . . . . .
. . . . . . . .                 . . . . . . . .

body: 12 11 10 9 17             body: 13 12 11 10 9

13 entered, 17 left
```

One move: add at the front, remove at the back. Every frame of the game does this. A long snake must still move in $\Theta(1)$ time.

Q9-1

- The body is a std::vector<int>, head at index 0. A move is:

```
body.insert(body.begin(), new_head);   // the head enters a new cell
body.pop_back();                       // the tail leaves
```

- The snake is $n = 1{,}000{,}000$ cells long. How many elements does one move shift?

A. 0
B. 1
C. About 1,000
D. About 1,000,000
E. None of the above

Memory representation of arrays

Address	Value
–	
0x0A08	var1
0x0A0C	var2
0x0A10	A
0x0A14	
0x0A18	
0x0A1C	
0x0A20	
0x0A24	
0x0A28	
0x0A2C	
0x0A30	
0x0A34	
0x0A38	
0x0A3C	
0x0A40	
0x0A44	
0x0A48	
0x0A4C	
0x0A50	
0x0A54	
–	

```
int var1;
int var2;
int A[7];
```

address of $A[i]$ = base + $i \times$ sizeof(T)

Address of A[4]?

*The price of that formula: elements must stay **side by side***

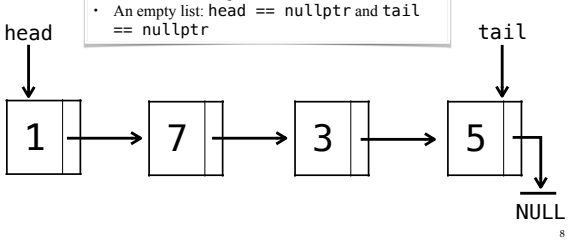
Linked lists

Linked lists

- A **linked list** stores each element in ints own **node**, anywhere in memory
  - a node holds the data and a pointer to the next node
  - the list keeps a pointer to the first node, the head, and usually to the last, the tail
- Operations (same as vectors)
  - **insert**: at the front, rear, specific index
  - **delete**: from the front, rear, specific index
  - **search**: find a node containing a specific value
  - **get**: retrieve the value at a specific position
  - **traverse**: sequentially "visit" each node in the list

Singly-linked list

- **Vocabulary**
  - **singly linked**: each node points to the next only, so we can walk from head to tail, never back
  - **length**: the number of nodes
  - the **tail** pointer is optional, but it makes adding at the end cheap
- An empty list: `head == nullptr` and `tail == nullptr`



Memory representation of linked lists

Address	Value	
…		
0x0A08	–5	
0x0A0C	0x0A20	
0x0A10		
0x0A14		
0x0A18	33	
0x0A1C	0x0000	
0x0A20	6	
0x0A24	0x0A48	
0x0A28		
0x0A2C		
0x0A30	21	
0x0A34	0x0A18	
0x0A38	3	
0x0A3C	0x0A08	
0x0A40		
0x0A44		
0x0A48	18	
0x0A4C	0x0A30	
0x0A50	0x0A38	**head**
0x0A54	0x0A18	**tail**
…		

Start at `head = 0x0A70` in the table and follow the `next` pointers. What elements are stored in this linked list?

Empty rows are memory used by something else


```
class SLList {
    private:
        struct Node {
            int   data;
            Node* next;
            Node(int value) : data(value), next(nullptr) {}
        };
        Node*  m_head;
        Node*  m_tail;
        size_t m_size;

    public:
        SLList() : m_head(nullptr), m_tail(nullptr), m_size(0) {}
        ~SLList() { clear(); }
        SLList(const SLList&) = delete;
        SLList& operator=(const SLList&) = delete;

        size_t size() const  { return m_size; }
        bool   empty() const { return m_size == 0; }
        int& front();
        int& back();
        void push_front(int value);
        void pop_front();
        void push_back(int value);
        void pop_back();
        void clear();
        void print() const;
        bool contains(int value) const;
        int& at(size_t index);
};
```


Walking the list

```
void SLList::print() const { // O(n)
    for (Node* p = m_head; p != nullptr; p = p->next)
        std::cout << p->data << ' ';
    std::cout << '\n';
}

bool SLList::contains(int value) const { // O(n)
    for (Node* p = m_head; p != nullptr; p = p->next)
        if (p->data == value)
            return true;
    return false;
}

int& SLList::at(size_t index) { // O(n)
    if (index >= m_size)
        throw std::out_of_range("at: bad index");
    Node* p = m_head;
    for (size_t i = 0; i < index; ++i)   // walk index steps
        p = p->next;
    return p->data;
}
```


Q9-2

- A student sums a list the way they would sum a vector

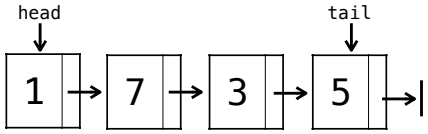
```
long total = 0;
for (size_t i = 0; i < list.size(); ++i)
    total += list.at(i);
```

- The list has $n = 100{,}000$ nodes. About how many `next` steps does this loop take?

A. About 10^5
B. About 2×10^5
C. About 5×10^9
D. About 10^{10}
E. None of the above

push_front(10)

Make the new node point to the old first node, then move the head




```
void SLList::push_front(int value) { // O(1)
    Node* n = new Node(value);
    n->next = m_head;       // link the new node to the old first node
    m_head = n;             // only then move head
    if (m_tail == nullptr)  // the list was empty
        m_tail = n;
    ++m_size;
}
```


Q9-3

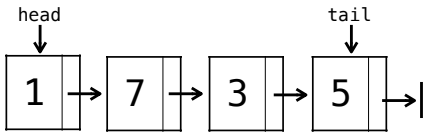
- Someone swaps the two steps

```
Node* n = new Node(value);
m_head = n;
n->next = m_head;
```

- The list was `1 7 3`, and we call `push_front(10)`. What does `print()` do next?

A. Prints "10 1 7 3"
B. Prints "10"
C. Prints "10 10 10 10 …" forever
D. Does not compile
E. None of the above

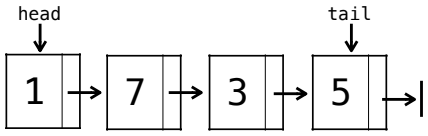
pop_front()




```
void SLList::pop_front() { // O(1)
    if (empty())
        throw std::underflow_error("pop_front of empty list");
    Node* old = m_head;
    m_head = m_head->next;
    delete old;
    if (m_head == nullptr)
        m_tail = nullptr;
    --m_size;
}
```


push_back(10)

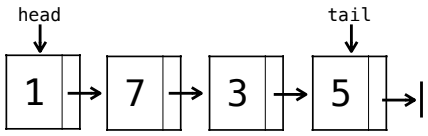
With a tail pointer, the end of the list is one step away. Without `m_tail` *we would walk the whole list first:* $\Theta(n)$




```
void SLList::push_back(int value) { // O(1)
    Node* n = new Node(value);
    if (m_tail == nullptr) {   // empty: the node is head and tail
        m_head = m_tail = n;
    } else {
        m_tail->next = n;      // link after the old tail
        m_tail = n;
    }
    ++m_size;
}
```

pop_back()

To remove the tail, the node before it must become the new tail. We must walk.




```
void SLList::pop_back() { // O(n)
    if (empty())
        throw std::underflow_error("pop_back of empty list");
    if (m_head == m_tail) {     // one node
        delete m_head;
        m_head = m_tail = nullptr;
    } else {
        Node* p = m_head;
        while (p->next != m_tail)   // walk to the node before the tail
            p = p->next;
        delete m_tail;
        m_tail = p;
        m_tail->next = nullptr;
    }
    --m_size;
}
```

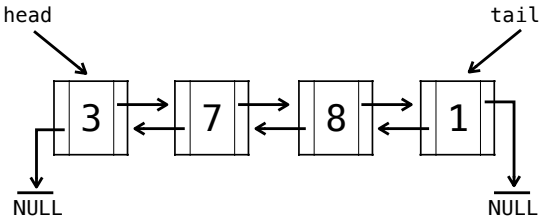

Q9-4

- An *SLList* keeps both `head` and `tail`. Which operation is still $\Theta(n)$?

A. `push_front`
B. `push_back`
C. `pop_front`
D. `pop_back`
E. None of the above

Doubly linked list

- Give every node a second pointer, to the **previous** node
  - cost: one more pointer per node
- The tail now knows its predecessor: `pop_back` is $\Theta(1)$
- Given a pointer to any node, removing it is $\Theta(1)$



Sentinels

- Every operation so far had special cases
  - ✓ empty list, one node, first node, last node
- Could avoid them with two dummy nodes, **sentinels**, that hold no data
  - ✓ the **header** sits before the first element, the **trailer** after the last
  - ✓ they are created by the constructor and never removed



  - ✓ every real node now has a real node or a sentinel on **both** sides
    - insert and remove are the same code everywhere: front, back, middle

```
class DLList {
    private:
        struct Node {
            int   data;
            Node* prev;
            Node* next;
            Node(int value = 0) : data(value), prev(nullptr), next(nullptr) {}
        };

        Node*  m_header;    // sentinel before the first element
        Node*  m_trailer;   // sentinel after the last element
        size_t m_size;

        void add_before(Node* v, int value);   // insert a new node before v
        void remove(Node* v);                  // unlink and free v

    public:
        DLList() : m_header(new Node), m_trailer(new Node), m_size(0) {
            m_header->next = m_trailer;
            m_trailer->prev = m_header;
        }
        ~DLList() {
            while (!empty()) pop_front();
            delete m_header;
            delete m_trailer;
        }
        // copy constructor and assignment deleted, as in SLList
        // other public methods are declared here
};
```



```
void DLList::add_before(Node* v, int value) {
    Node* u = v->prev;           // the node before v
    Node* n = new Node(value);
    n->prev = u;   // the new node points to its neighbors
    n->next = v;
    u->next = n;   // the neighbors point to the new node
    v->prev = n;
    ++m_size;
}

void DLList::remove(Node* v) {
    Node* u = v->prev;
    Node* w = v->next;
    u->next = w;   // bypass v in both directions
    w->prev = u;
    delete v;
    --m_size;
}
```



```
void push_front(int value) {
    add_before(m_header->next, value);
}

void push_back(int value)  {
    add_before(m_trailer, value);
}

void pop_front() {
    if (empty()) throw std::underflow_error("pop_front of empty list");
    remove(m_header->next);
}

void pop_back() {
    if (empty()) throw std::underflow_error("pop_back of empty list");
    remove(m_trailer->prev);
}
```


Q9-5

- A *DLList* holds “3 7 8”. We run:

```
add_before(m_trailer->prev, 5);
remove(m_header->next);
```

- What does the list hold now, front to back?

A. 7 8 5
B. 7 5 8
C. 3 7 5
D. 5 7 8
E. None of the above

The snake, again

- The snake on a DLList
  - ✓ `push_front` and `pop_back`, both Θ(1)

```
void step(DLList& body, int cell, bool ate) {
    body.push_front(cell);    // the head enters the new cell
    if (!ate)body.pop_back(); // the tail leaves
}
```

snake length	vector	SLList	DLList
10^3	0.036 us	0.99 us	0.042 us
10^4	0.53 us	9.7 us	0.042 us
10^5	8.0 us	113 us	0.043 us
10^6	89 us	1,481 us	0.043 us

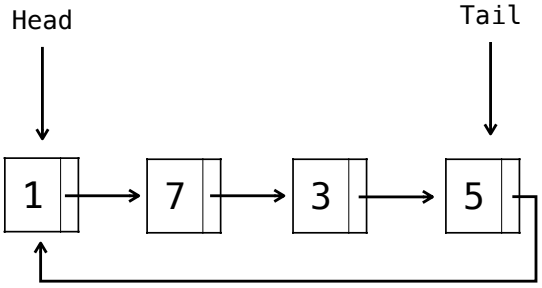
One move of a million-cell snake: 34,000x faster than on the SLList, and the time does not depend on the length. Intel Core i7-8700B, clang -O2, 1,000 moves per length, median of three runs.

Q9-6

- Fill in the table with the worst case cost. Both lists keep a tail pointer; "index *i*" means you are given *i*, not a pointer to a node

operation	dynamic array	singly linked	doubly linked
add at the end			
remove from the end			
add at the front			
insert at index *i*			
remove at index *i*			
read or write element *i*			
insert after a node you hold			

Circular singly linked list



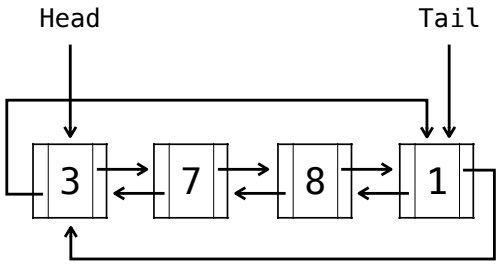
Make the tail point back to the head: the list becomes a ring. Natural for anything that takes turns: e.g., round-robin scheduling, multiplayer turns, a playlist on repeat.

Q9-7

- n = 7 players, numbered 1 to 7, and every k = 3rd player is out, counting from player 1. Who survives?

A. 1
B. 3
C. 4
D. 7
E. None of the above

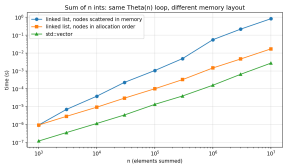
Circular doubly linked list

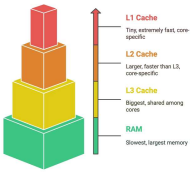


A doubly linked ring: from any node, walk either way, forever.

Linked lists and the hardware

- Contiguous memory makes scanning fast, because of the memory hierarchy (cache blocks)
- Sum n integers three ways:





Intel Core i7-8700B, clang -O2, median of three runs

- At $n = 10^7$: vector **0.0027 s**, list in allocation order **0.017 s**, list in shuffled order **0.84 s**
  - every version is $\Theta(n)$; the shuffled list is about **300x** slower than the vector
  - this is why the *SLList* snake lost to the vector

STL containers

Collection	Description	Implementation	Random Access	Insertion/Deletion	Memory per element
`std::array`	fixed-size array	contiguous memory	O(1)	Not designed for insertion/deletion	none
`std::vector`	dynamic array	contiguous memory	O(1)	O(n) at arbitrary positions; O(1) amortized at end	low
`std::deque`	double-ended queue	segmented array	O(1)	O(1) at both ends; O(n) in middle	medium
`std::list`	doubly-linked list	non-contiguous nodes	O(n)	O(1) at a given position; O(n) to find position	high
`std::forward_list`	singly-linked list	non-contiguous nodes	O(n)	O(1) after a given position; O(n) to find position	medium